# Dynamic Sparse Training with Structured Sparsity

Mike Lasby[1], Anna Golubeva[2,3], Utku Evci[4], Mihai Nica[5,6], Yani A. Ioannou[1]

[1]University of Calgary [2]Massachusetts Institute of Technology [3]IAIFI [4]Google Research [5]University of Guelph [6]Vector Institute for AI

## Motivation

**Unstructured Sparse Neural Networks (SNNs) are hard to accelerate!**

- State-of-the-art Dynamic Sparse Training (DST) methods learn *unstructured* SNNs with 85–95% fewer weights than dense models, while maintaining similar generalization.
- Sparse training methods employ sparsity *both during training and inference*, unlike pruning and other methods [5] that only exploit sparsity at inference time.
- While unstructured SNNs enable a large reduction in Floating Point Operations (FLOPs) in theory, realizing these speedups on hardware is challenging.
- Structured sparse pruning often results in worse generalization than unstructured SNNs.

## Method

1. We propose a novel sparse-to-sparse DST method, Structured RigL (SRigL), based on RigL [1]. SRigL learns a SNN with constant fan-in structured sparsity while maintaining generalization comparable with RigL up to a high sparsity level (99%) for a variety of network architectures. This structure is a particular case of "N:M sparsity" [4].
2. We show that at sparsity levels >90% RigL ablates whole neurons. By allowing ablation in SRigL, we match the generalization performance of RigL even in this high-sparsity regime.
3. We motivate our choice of structure with a theoretical analysis of SNN output norm variance and find that the constant fan-in constraint does not have a negative effect.
4. We demonstrate that our constant fan-in sparsity enables a compact representation that is not only parameter- and memory-efficient, but also amenable to real-world acceleration.

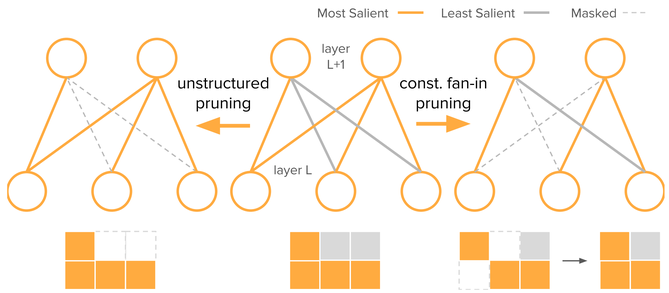


**Figure 1.** Constant fan-in pruning v.s. unstructured pruning.

## Constant Fan-In Theoretical Analysis

- Smaller output variance is indicative of training stability
- We derive exact expressions for the variance of the layer output norm at initialization
- Constant fan-in consistently yields smaller variance than other sparsity types

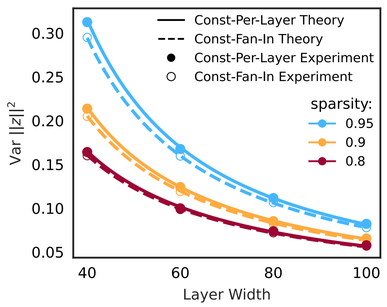


**Figure 2.** Output-norm variance analysis.

## Results

**Table 1. Top-1 ImageNet test accuracy of ResNet-50 trained with a variety of DST methods**, highlighting methods that both are sparse-to-sparse *and* learn structured sparsity similar to SRigL — only DSB-16 (2:4 and 1:4 sparsity) is directly comparable in this regard. RigL and SRigL results are from our experiments, other values are obtained from each method's corresponding paper, U.N.O.

| | training | | sparsity | | | | |
|---|---|---|---|---|---|---|---|
| method | method | structured | 50% | 75% | 80% | 90% | 93.75% |
| Static* | sparse | no | – | – | $70.6 \pm 0.06$ | $65.8 \pm 0.04$ | – |
| SET* | sparse | no | – | – | $72.9 \pm 0.39$ | $69.6 \pm 0.23$ | – |
| DeepR§ | sparse | no | – | – | 71.7 | 70.2 | – |
| DSR | sparse | no | – | – | 73.3 | 71.6 | – |
| Top-KAST‡ | sparse | no | – | – | 74.76 | 70.42 | – |
| MEST† | sparse | no | – | – | **75.39** | 72.58 | – |
| RigL | sparse | no | – | – | 74.98 | **72.81** | – |
| DSB-16 | **sparse** | **yes** | 76.33 | 74.04 | – | – | – |
| SRigL (Ours) | **sparse** | **yes** | **76.60** | **75.55** | 75.01 | 72.71 | 70.56 |
| SNFS (ERK)* | dense | no | – | – | $75.2 \pm 0.11$ | $73.0 \pm 0.04$ | – |
| SR-STE | dense | yes | – | 76.2 | – | – | 71.5 |
| *dense ResNet-50:* | | | | | 76.7 | | |

* values obtained from Evci et al. [1]
§ values obtained from Mostafa & Wang [3]
† values for the MEST (x0.67+EM) variant, matched to the same number of training FLOPs as RigL
‡ values tabulated for Top-K Always Sparse Training (Top-KAST) correspond to the *backwards sparsity* as Top-KAST uses different sparsities in the forward and backward passes. For 80% sparsity, the value reported is based on using a random regrow criteria, unlike the 90% sparsity value. For more information see Table 1 in [2]

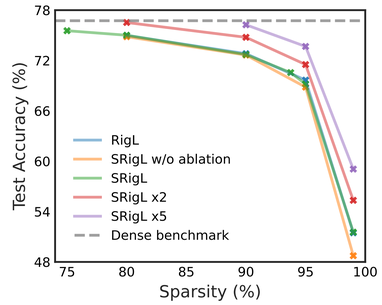


**Figure 3.** ResNet-50/ImageNet

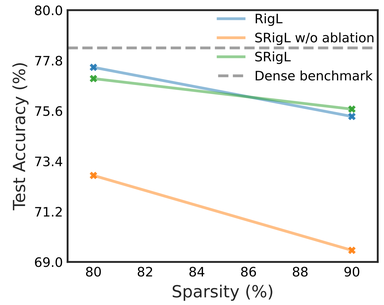


**Figure 4.** Vision Transformer (ViT-B/16)/ImageNet

## FLOPs & Acceleration

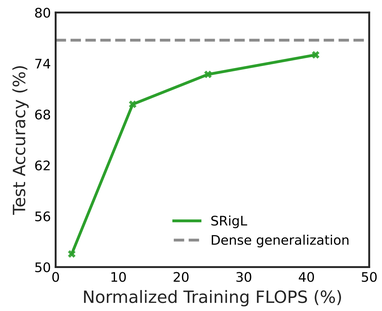


**Figure 5. Training FLOPs** for SRigL on ResNet-50/ImageNet at a variety of sparsities compared with dense generalization. FLOPs are normalized by dense training FLOPs.

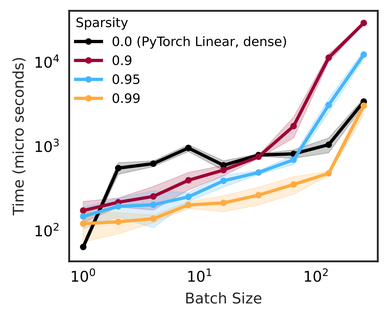


**Figure 6. PyTorch timings.** Benchmarking a naive (non-optimized) PyTorch implementation of our condensed representation on CPU.

## Neuron Ablation

- At high sparsities, RigL ablates large numbers of neurons. Effectively, *RigL reduces the width of the model at high sparsities*.
- Under a strict constant fan-in constraint, neurons can never be ablated. Leading to very sparely connected neurons at high sparsities.
- We implement a **neuron ablation method**, allowing SRigL to maintain both a constant fan-in constraint *and* to reduce layer width at high sparsities.


**Figure 7. Neuron ablation.** At sparsity levels over 90%, RigL learns to completely mask (ablate) a large number of neurons within each layer, effectively reducing layer width. Imposing a constant fan-in constraint requires all neurons to have the same number of (non-pruned) incoming weights and therefore inhibits ablation, which results in worse generalization performance than RigL. Allowing SRigL to ablate neurons restores RigL-level performance.

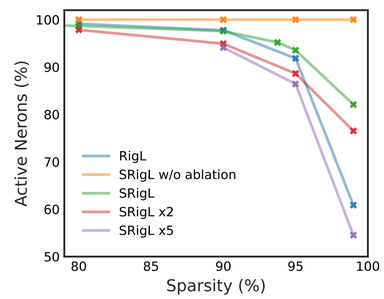


**Figure 8.** Neuron Ablation

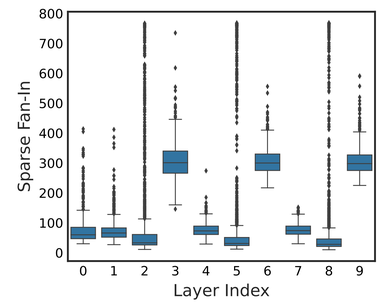


**Figure 9. Sparse Fan-In vs. ViT-B/16 layer index at the end of training with RigL at 90% sparsity.** Only the first 10 layers are shown for clarity. We find that RigL learns a sparse connectivity with large variance in fan-in between neurons within the same layer with some neurons receiving up to $\times 10$ the number of active connections than the mean for the same layer.

## Acknowledgements

We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), Alberta Innovates, the Digital Research Alliance of Canada, the NSF AI Institute for Artificial Intelligence and Fundamental Interactions (IAIFI). We are grateful for computational resources made available to us by Google and Denvr Dataworks. We also acknowledge the very helpful feedback of Trevor Gale.

## References

[1] Evci, U., Gale, T., Menick, J., Castro, P. S., and Elsen, E. Rigging the Lottery: Making All Tickets Winners. Technical Report arXiv:1911.11134, arXiv, July 2021. arXiv:1911.11134.

[2] Jayakumar, S., Pascanu, R., Rae, J., Osindero, S., and Elsen, E. Top-KAST: Top-K Always Sparse Training. In *Advances in Neural Information Processing Systems*, volume 33, pp. 20744–20754. Curran Associates, Inc., 2020.

[3] Mostafa, H. and Wang, X. Parameter efficient training of deep convolutional neural networks by dynamic sparse reparameterization. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 4646–4655. PMLR, May 2019. ISSN: 2640-3498.

[4] Nvidia. Nvidia A100 Tensor Core GPU Architecture. Technical report, Nvidia, 2020.

[5] Zhou, A., Ma, Y., Zhu, J., Liu, J., Zhang, Z., Yuan, K., Sun, W., and Li, H. Learning n:m fine-grained structured sparse neural networks from scratch. In *International Conference on Learning Representations*, 2021.